

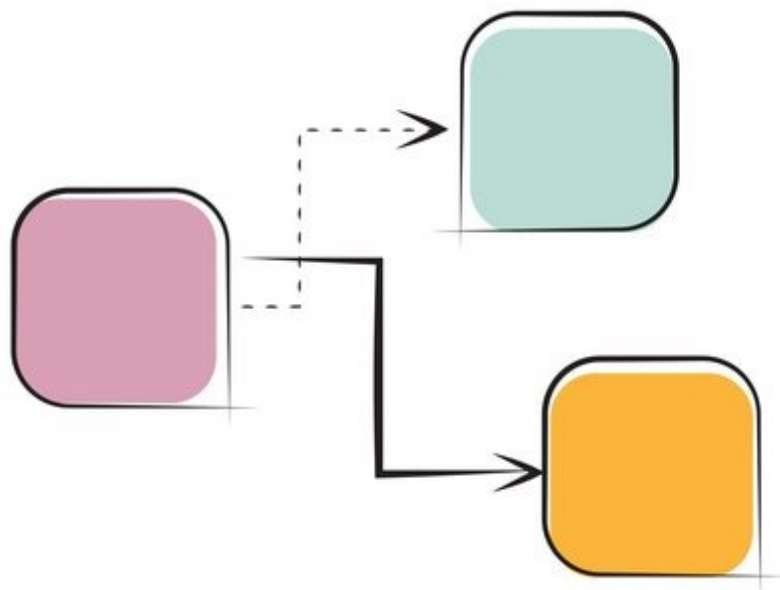


Universitat
Oberta
de Catalunya

Análisis y Diseño con Patrones

Ejercicios iniciales de repaso

Colección de ejercicios de repaso sobre los principales diagramas UML que se utilizarán en la asignatura ADP y su notación.



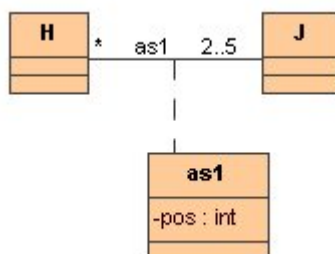
Equipo docente de Análisis y Diseño con Patrones
Última actualización: Febrero 2018

Ejercicio 1: Notación UML

El lenguaje UML (Unified Modeling Language) es el lenguaje estándar para crear modelos visuales de software.

Representar gráficamente, en UML, las siguientes situaciones:

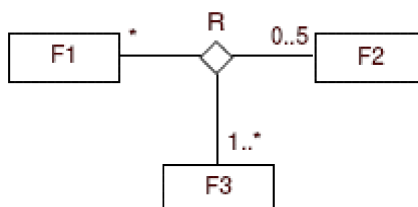
- a) Tenemos dos clases: *H* y *J* asociadas a través de la asociación *as1*. Cada instancia de *H* puede estar asociada a mínimo 2 y máximo 5 instancias de *J*. Cada instancia de *J* puede estar asociada a un número indeterminado de instancias de *H*. Cada vez que una instancia de *H* y *J* estén asociadas, queremos saber cuál es la posición relativa de aquella instancia de *J* de entre todas las que están asociadas a *H* (para ello utilizaremos un atributo llamado *pos* de tipo entero).



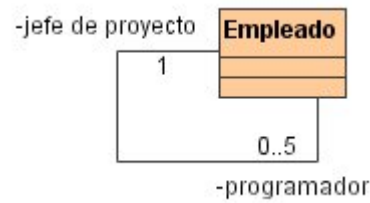
- b) Tenemos dos clases: *K* y *L*. Cada instancia de *L* pertenece a una instancia de *K* hasta al punto de que si borramos su instancia de *K* del sistema, borraremos también la de *L*. Cada instancia de *K* puede tener un máximo de 19 instancias de *L*.



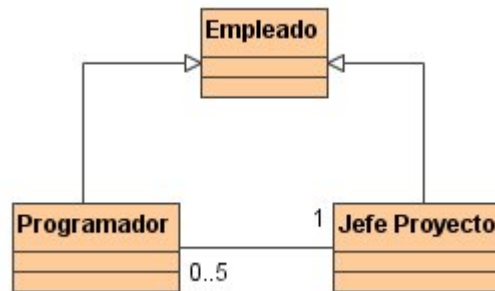
- c) Definimos una asociación ternaria *R* entre las clases *F1*, *F2* y *F3*. Una pareja cualquiera de instancias de las clases *F1* y *F2* tiene que estar asociada como mínimo con una instancia de *F3*. Para cada par de instancias de *F1* y *F3* existen como máximo 5 instancias de *R*. Para las parejas de instancias de *F2* y *F3* no hay ninguna restricción en cuanto a la cardinalidad de *R*.



- d) Cada empleado de una empresa de Informática puede tener los roles de programador o jefe de proyecto. En el papel de jefe de proyecto, podrá dirigir hasta 5 programadores o ninguno. Cada programador tendrá un solo jefe de proyecto.



Otra posible solución sería:



Fijaros que si consideramos que un empleado puede ser al mismo tiempo programador y jefe de proyecto, esta solución implica la existencia de clasificación múltiple. La clasificación múltiple permite que un mismo objeto (empleado en este caso) sea al mismo tiempo instancia de la clase Programador y de la clase *Jefe Proyecto*. Esto no lo admiten los lenguajes de programación OO por lo que en la etapa de diseño este diagrama de clases tendría que transformarse utilizando patrones de diseño adecuados.

Ejercicio 2: Modelo conceptual de datos (diagrama estático de análisis)

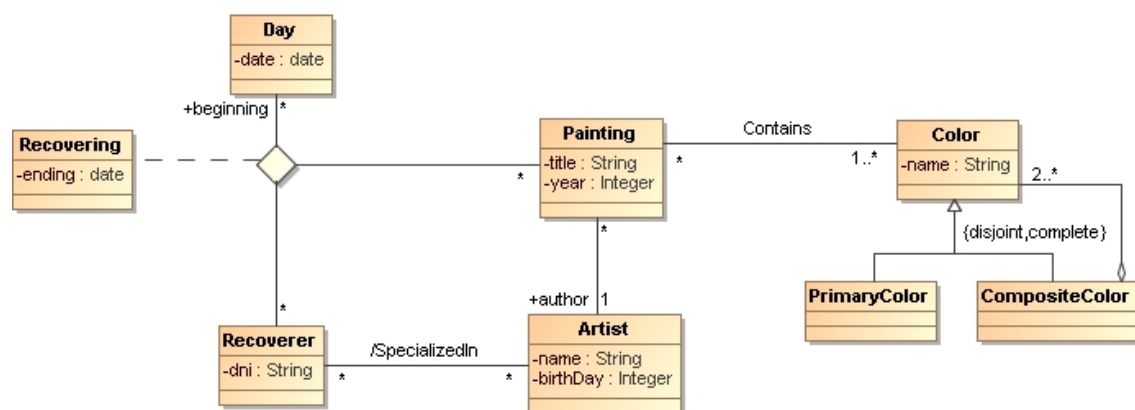
El diagrama de clases (*class diagram*), también llamado modelo conceptual de datos o diagrama estático, es, probablemente, el diagrama más conocido de UML. Sirve para modelar las clases de objetos y sus relaciones, poniendo énfasis en los aspectos estructurales por encima del comportamiento.

Queremos crear un sistema que nos permita llevar un control sobre las restauraciones de cuadros que se hacen en un museo. De cada cuadro queremos saber el título, el año en que se pintó y su autor. Los cuadros se identifican por el nombre del cuadro y el nombre del pintor. De los autores queremos conocer el nombre y el año de nacimiento.

Cada cuadro contiene un conjunto de colores, de los que queremos saber el nombre, y que pueden ser primarios o compuestos. De los colores compuestos queremos conocer cuáles son los colores que los forman (que pueden ser a su vez también compuestos).

Cada cuadro puede ser restaurado diversas veces y por diversos restauradores, de los cuales queremos saber el dni. También queremos conocer cuáles son los pintores en que un restaurador es especialista (que son los pintores de los cuales ha restaurado más de un cuadro). Un restaurador puede participar en diversas restauraciones de un mismo cuadro. De cada una de estas participaciones queremos saber las fechas de inicio y de final. Puede suceder que un restaurador esté trabajando en diversos cuadros diferentes en una misma fecha. Por otra parte, en una misma fecha diversos restauradores pueden estar trabajando en el mismo cuadro.

Haced el diagrama de clases correspondiente.



NOTA: Las visibilidades a atributos y roles de asociación no tendrían que aparecer, ya que se trata de un diagrama de análisis. Sólo son debidos a una limitación de la herramienta.

NOTA: Fijaos que la fecha de inicio tiene que participar en la relación Restauración (*Recovering*), ya que si no, un restaurador no podría restaurar dos veces el mismo cuadro (hecho que se tiene que permitir, según el enunciado). El motivo es que, por la propia definición de asociación, no puede haber dos instancias de una asociación con los mismos participantes, es decir, si no hiciéramos que la fecha de inicio participara en la ternaria, no podría haber dos instancias de Restauración con los mismos Cuadro y Restaurador. Para permitir instancias con estos dos extremos repetidos, tenemos que añadir la clase

Fecha como participante de la asociación. La fecha de finalización, en cambio, es suficiente que sea un atributo de la clase Restauración.

NOTA: Falta explicitar todas aquellas restricciones y reglas de derivación que no se pueden expresar de forma gráfica.

Por ejemplo:

- Un restaurador es especialista en un pintor cuando hay más de una instancia de *Recovering* que relaciona al restaurador con una obra del pintor (asociación derivada *SpecializedIn*).
- No puede haber dos instancias de *Painting* con el mismo nombre si están asociadas al mismo pintor (restricción de unicidad de *Painting*).

Ejercicio 3: Diagrama de casos de uso

El diagrama de casos de uso (*use case diagram*) describe las relaciones y dependencias entre los *casos de uso* y los actores que participan en ellos. El diagrama de casos de uso complementa, pero en ningún caso sustituye, la descripción textual de los casos de uso, ya que no incluye información sobre cuál es el comportamiento del sistema.

Se quiere de informatizar la gestión de la policía municipal de un ayuntamiento, donde hay los siguientes cargos:

El cabo, que tiene asignadas las siguientes tareas:

- Dar de alta los policías. De cada policía da los apellidos y nombre, el NIF, el sexo, la fecha de nacimiento, un texto sobre formación recibida, y otro sobre experiencia laboral anterior.
- Crear el informe diario de las patrullas en coche, con la fecha del día y la primera anotación del día. Las anotaciones tienen un número correlativo, los apellidos y nombre del autor y el texto.

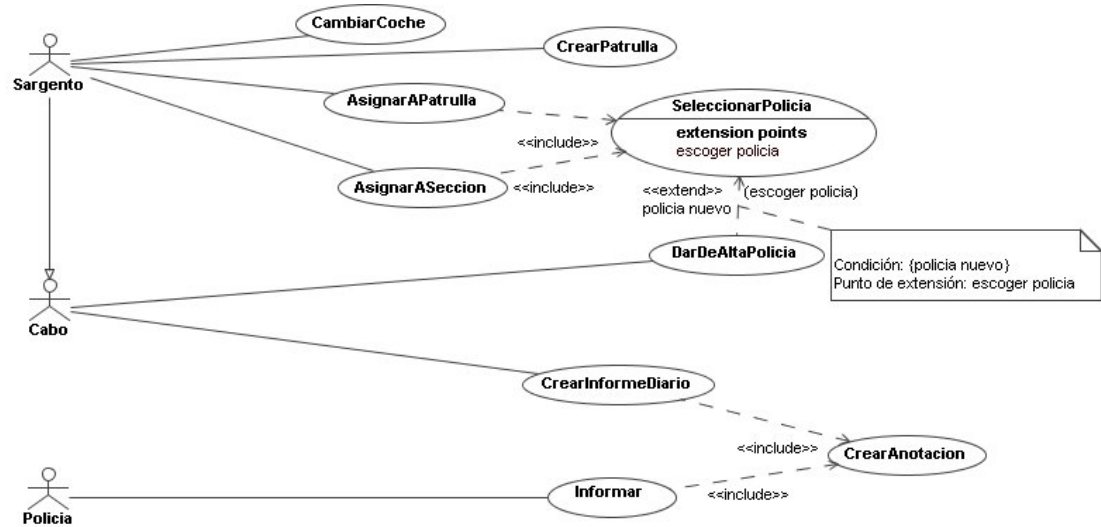
El sargento, que tiene asignadas las siguientes tareas:

- Crear patrullas. Cada patrulla tiene un código y puede ser en coche o sin. Cuando crea una patrulla en coche le asigna un coche, que selecciona de una lista de matrículas de coches disponibles. También puede cambiar el coche asignado a una patrulla.
- Asignar trabajo a los policías; les puede asignar dos tipos de trabajos: o bien asignarlos a una patrulla o bien asignarlos a una sección (son dos funciones independientes). Las patrullas pueden tener hasta cuatro policías. Cuando asigna un policía a una sección tiene que describir la función que hará. Cada policía que asigna a un trabajo - ya sea a una patrulla o a una sección- o bien lo escoge de una lista de apellidos y nombres de policías o bien lo da de alta en aquel momento. En todo momento un policía sólo puede estar asignado a un trabajo como máximo; por lo tanto cuando se lo asigna a un trabajo se borra la asignación anterior, si tenía.
- También puede hacer las funciones del cabo.

El policía, que tiene asignadas las siguientes tareas:

- Informar. Consiste en introducir su NIF, el código de su patrulla y la matrícula del coche asignado a ésta y acto seguido crear una anotación, de la cual el número se asignará de manera correlativa, los apellidos y nombre del autor los obtendrá el sistema a partir del NIF introducido y el texto lo introducirá él mismo.

a) Haced el diagrama de casos de uso.



b) Haced la especificación textual del o de los casos de uso que intervienen en la asignación de un policía a un trabajo, descompuestos en pasos numerados, de cada uno de los cuales se indicará en especial si lo hace el sistema o bien un actor y qué datos intervienen.

Caso de uso Seleccionar Policía

Resumen de la funcionalidad: asignar un trabajo a un policía

Actores: **Sargento**

Casos de uso relacionados: DarDeAltaPolicia, AsignarAPatrulla, AsignarASeccion,

Precondición: hay al menos una patrulla con menos de 4 policías asignados o una sección.

Postcondición: el policía seleccionado o dado de alta está asignado a la patrulla o sección seleccionada.

Proceso normal principal:

1. El sistema lee de la base de datos todos los policías.
2. El sistema presenta los apellidos y nombres de los policías leídos al paso anterior y da opción al sargento a escoger a uno o a darlo de alta.
3. El **Sargento** selecciona a uno de los policías.

Alternativas de proceso y excepciones:

- 2a. El **Sargento** elige el opción de dar de alta un policía, en lugar de hacer el paso 3:
 - 2a1. El sistema ejecuta el caso de uso DarDeAltaPolicia.

Caso de uso AsignarAPatrulla

Resumen de la funcionalidad: asignar un policía a una patrulla

Actores: **Sargento**

Casos de uso relacionados: SeleccionarPolicia

Precondición: hay al menos una patrulla con menos de 4 policías asignados.

Postcondición: el policía seleccionado o dado de alta está asignado a la patrulla seleccionada.

Proceso normal principal:

1. El sistema ejecuta el caso de uso SeleccionarPolicia.
2. El sistema lee de la base de datos todas las patrullas que tienen menos de 4 policías.
3. El sistema presenta el código de las patrullas leídas al paso anterior y da opción al **Sargento** a seleccionar una.
4. El **Sargento** selecciona una de las patrullas presentadas y confirma la selección.
5. El sistema borra de la base de datos la asignación anterior del policía a una sección o patrulla.
6. El sistema graba en la base de datos la asignación del policía a la patrulla seleccionada.

Alternativas de proceso y excepciones:

- 2a. No hay ninguna patrulla con menos de 4 policías:
 - 2a1. El sistema cancela el caso de uso.

Caso de uso AsignarASeccion

Resumen de la funcionalidad: asignar un policía a una sección

Actores: **Sargento**

Casos de uso relacionados: SeleccionarPolicia

Precondición: hay al menos una sección.

Postcondición: el policía seleccionado o dado de alta está asignado a la sección seleccionada.

Proceso normal principal:

1. El sistema ejecuta el caso de uso SeleccionarPolicia.
2. El sistema lee de la base de datos todas las secciones.
3. El sistema presenta el nombre de las secciones leídas en el paso anterior y da opción al sargento a seleccionar una y a introducir la función.
4. El **Sargento** selecciona una de las secciones presentadas, introduce la función y confirma la selección.
5. El sistema borra de la base de datos la asignación anterior del policía a una sección o patrulla.
6. El sistema graba en la base de datos la asignación del policía a la sección seleccionada con la función introducida.

Alternativas de proceso y excepciones:

Ninguna.

Caso de uso DarDeAltaPolicia

Resumen de la funcionalidad: añadir un policía a la base de datos

Actores: **Cabo** y por herencia **Sargento**

Casos de uso relacionados: SeleccionarPolicia

Precondición: ninguna.

Postcondición: el policía está en la base de datos.

Proceso normal principal:

1. El sistema pide al **Cabo** los datos del policía: los apellidos y nombre, el NIF, el sexo, la fecha de nacimiento, la formación recibida y la experiencia laboral anterior.
2. El **Cabo** introduce los datos pedidos y los confirma.
3. El sistema crea en la base de datos un policía con los datos introducidos.

Alternativas de proceso y excepciones:

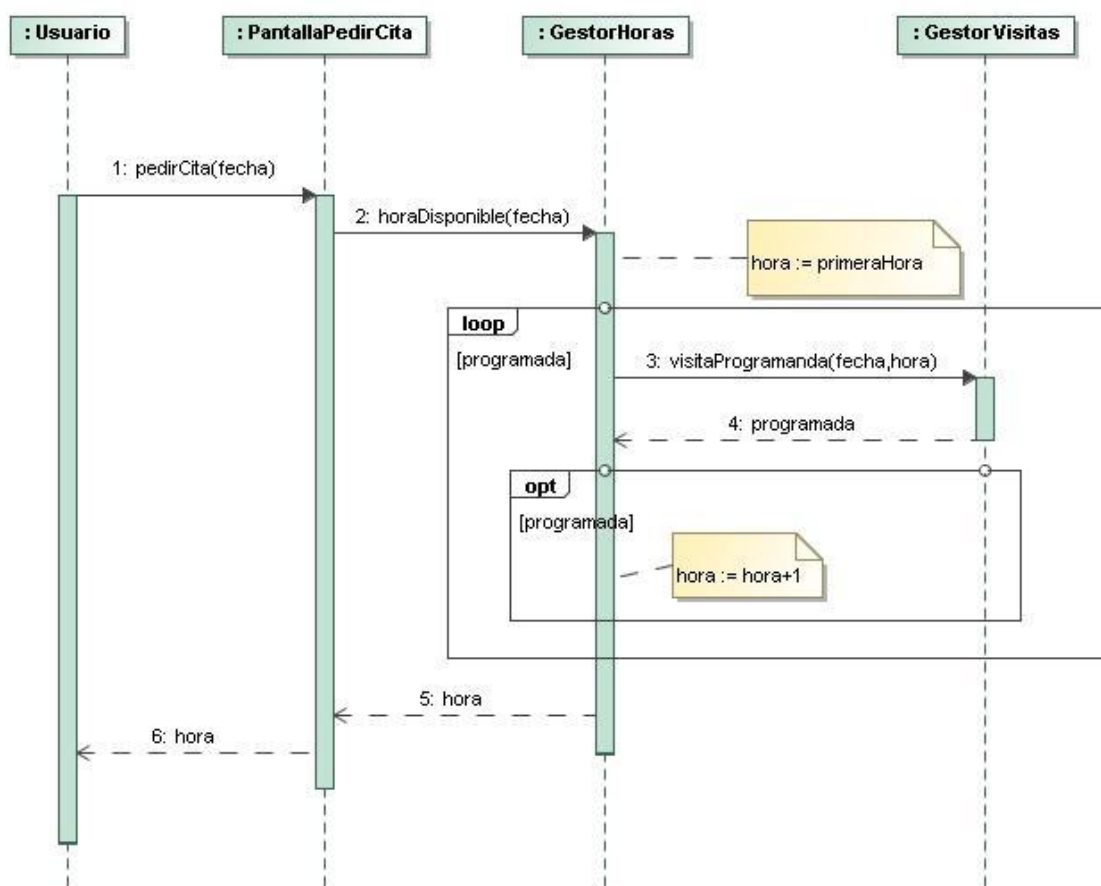
Ninguna.

Ejercicio 4: Diagrama de secuencia

El diagrama de secuencia (*sequence diagram*) modela la interacción entre diversos objetos poniendo énfasis en la secuencia temporal (es decir, en el orden en que interactúan los objetos).

Queremos diseñar un sistema de gestión de citas de pacientes en un centro médico. Cuando un usuario pide una cita indica, en la pantalla de pedir cita, en qué fecha quiere la cita. Esta pantalla llamará a la operación *horaDisponible(fecha)* de la instancia del gestor de horas que, a su turno, hará lo siguiente: irá al gestor de visitas para pedirle si ya hay una visita programada a primera hora. Si ya hay una visita programada, probará la siguiente hora hasta que encuentre una hora para la cual no haya ninguna visita programada para la fecha dada.

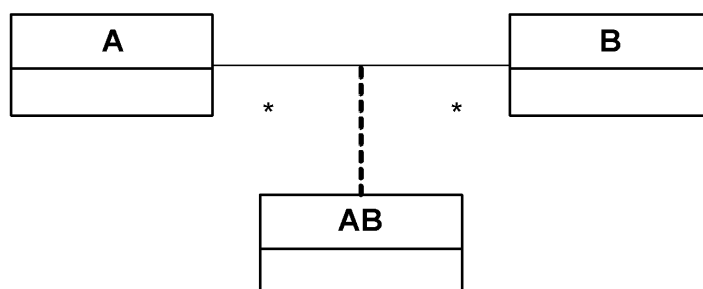
Haz un diagrama de secuencia que represente la situación descrita.



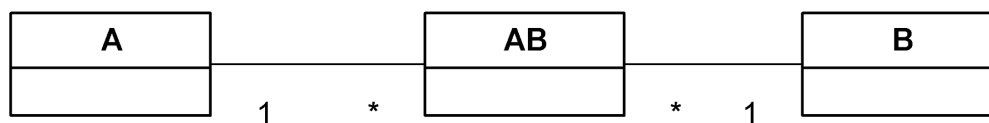
Ejercicio 5: Normalización de clases asociativas

El concepto de clase asociativa no existe en los lenguajes de programación, de modo que las clases asociativas deben convertirse en clases estándares antes de programarse. Este proceso se conoce con el nombre de normalización o reificación de clases asociativas.

Dado el siguiente modelo:



Representa gráficamente un modelo equivalente utilizando únicamente asociaciones binarias 1 a * y sin utilizar ninguna clase asociativa.



Restricción de integridad: Una instancia de AB sólo puede asociarse como máximo una vez a la misma pareja de A y B (es decir, no pueden existir dos instancias de AB relacionadas con el mismo par de instancias de A y B).

* La restricción se ha expresado textualmente ya que UML no dispone de notación gráfica para expresarla. Podría definirse formalmente utilizando el lenguaje OCL (Object Constraint Language).